JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Reader

java.lang.Object
    java.io.Reader

All Implemented Interfaces:

Closeable, AutoCloseable, Readable

Direct Known Subclasses:

BufferedReader, CharArrayReader, FilterReader, InputStreamReader, PipedReader, StringReader

---

```
public abstract class Reader
extends Object
implements Readable, Closeable
```

Abstract class for reading character streams. The only methods that a subclass must implement are read(char[], int, int) and close(). Most subclasses, however, will override some of the methods defined here in order to provide higher efficiency, additional functionality, or both.

Since:

1.1

See Also:

BufferedReader, LineNumberReader, CharArrayReader, InputStreamReader, FileReader, FilterReader, PushbackReader, PipedReader, StringReader, Writer

Field Summary

Fields

Modifier and Type	Field	Description
protected Object	**lock**	The object used to synchronize operations on this stream.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**Reader**()	Creates a new character-stream reader whose critical sections will

	synchronize on the reader itself.
protected **Reader**(Object lock)	Creates a new character-stream reader whose critical sections will synchronize on the given object.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
abstract void	**close**()	Closes the stream and releases any system resources associated with it.
void	**mark**(int readAheadLimit)	Marks the present position in the stream.
boolean	**markSupported**()	Tells whether this stream supports the mark() operation.
static Reader	**nullReader**()	Returns a new Reader that reads no characters.
static Reader	**of**(CharSequence cs)	Returns a Reader that reads characters from a CharSequence.
int	**read**()	Reads a single character.
int	**read**(char[] cbuf)	Reads characters into an array.
abstract int	**read**(char[] cbuf, int off, int len)	Reads characters into a portion of an array.
int	**read**(CharBuffer target)	Attempts to read characters into the

		specified character buffer.
String	**readAllAsString**()	Reads all remaining characters into a string.
List<String>	**readAllLines**()	Reads all remaining characters as lines of text.
boolean	**ready**()	Tells whether this stream is ready to be read.
void	**reset**()	Resets the stream.
long	**skip**(long n)	Skips characters.
long	**transferTo**(Writer out)	Reads all characters from this reader and writes the characters to the given writer in the order that they are read.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

lock

protected Object lock

The object used to synchronize operations on this stream. For efficiency, a character-stream object may use an object other than itself to protect critical sections. A subclass should therefore use the object in this field rather than this or a synchronized method.

Constructor Details

Reader

`protected Reader()`

Creates a new character-stream reader whose critical sections will synchronize on the reader itself.

Reader

`protected Reader(Object lock)`

Creates a new character-stream reader whose critical sections will synchronize on the given object.

Parameters:

`lock` - The Object to synchronize on.

Method Details

nullReader

`public static Reader nullReader()`

Returns a new `Reader` that reads no characters. The returned stream is initially open. The stream is closed by calling the `close()` method. Subsequent calls to `close()` have no effect.

While the stream is open, the `read()`, `read(char[])`, `read(char[], int, int)`, `read(CharBuffer)`, `ready()`, `skip(long)`, and `transferTo()` methods all behave as if end of stream has been reached. After the stream has been closed, these methods all throw `IOException`.

The `markSupported()` method returns `false`. The `mark()` and `reset()` methods throw an `IOException`.

The `object` used to synchronize operations on the returned `Reader` is not specified.

Returns:

a `Reader` which reads no characters

Since:

11

of

```
public static Reader of(CharSequence cs)
```

Returns a `Reader` that reads characters from a `CharSequence`. The reader is initially open and reading starts at the first character in the sequence.

The returned reader supports the `mark()` and `reset()` operations.

The resulting reader is not safe for use by multiple concurrent threads. If the reader is to be used by more than one thread it should be controlled by appropriate synchronization.

If the sequence changes while the reader is open, e.g. the length changes, the behavior is undefined.

Parameters:

`cs` - `CharSequence` providing the character stream.

Returns:

a `Reader` which reads characters from `cs`

Throws:

`NullPointerException` - if `cs` is `null`

Since:

24

read

```
public int read(CharBuffer target)
         throws IOException
```

Attempts to read characters into the specified character buffer. The buffer is used as a repository of characters as-is: the only changes made are the results of a put operation. No flipping or rewinding of the buffer is performed. If the length of the specified character buffer is zero, then no characters will be read and zero will be returned.

Specified by:

`read` in interface `Readable`

Parameters:

`target` - the buffer to read characters into

Returns:

The number of characters added to the buffer, possibly zero, or -1 if this source of characters is at its end

Throws:

`IOException` - if an I/O error occurs

`NullPointerException` - if target is null

`ReadOnlyBufferException` - if target is a read only buffer, even if its length is zero

Since:

1.5

read

```
public int read()
         throws IOException
```

Reads a single character. This method will block until a character is available, an I/O error occurs, or the end of the stream is reached.

Subclasses that intend to support efficient single-character input should override this method.

Returns:

The character read, as an integer in the range 0 to 65535 (`0x00-0xffff`), or -1 if the end of the stream has been reached

Throws:

`IOException` - If an I/O error occurs

read

```
public int read(char[] cbuf)
         throws IOException
```

Reads characters into an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached.

If the length of `cbuf` is zero, then no characters are read and `0` is returned; otherwise, there is an attempt to read at least one character. If no character is available because the stream is at its end, the value `-1` is returned; otherwise, at least one character is read and stored into `cbuf`.

Parameters:

`cbuf` - Destination buffer

Returns:

The number of characters read, or -1 if the end of the stream has been reached

Throws:

`IOException` - If an I/O error occurs

read

```
public abstract int read(char[] cbuf,
                         int off,
                         int len)
                  throws IOException
```

Reads characters into a portion of an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached.

If `len` is zero, then no characters are read and `0` is returned; otherwise, there is an attempt to read at least one character. If no character is available because the stream is at its end, the value `-1` is returned; otherwise, at least one character is read and stored into `cbuf`.

Parameters:

`cbuf` - Destination buffer

`off` - Offset at which to start storing characters

`len` - Maximum number of characters to read

Returns:

The number of characters read, or -1 if the end of the stream has been reached

Throws:

`IndexOutOfBoundsException` - If `off` is negative, or `len` is negative, or `len` is greater than `cbuf.length - off`

`IOException` - If an I/O error occurs

readAllLines

```
public List<String> readAllLines()
                           throws IOException
```

Reads all remaining characters as lines of text. This method blocks until all remaining characters have been read and end of stream is detected, or an exception is thrown. This method does not close the reader.

When this reader reaches the end of the stream, further invocations of this method will return an empty list.

A *line* is either a sequence of zero or more characters followed by a line terminator, or it is a sequence of one or more characters followed by the end of the stream. A line does not include the line terminator.

A *line terminator* is one of the following: a line feed character `"\n"` (U+000A), a carriage return character `"\r"` (U+000D), or a carriage return followed immediately by a line feed `"\r\n"` (U+000D U+000A).

The behavior for the case where the reader is *asynchronously closed*, or the thread interrupted during the read, is highly reader specific, and therefore not specified.

If an I/O error occurs reading from the stream then it may do so after some, but not all, characters have been read. Consequently the stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the reader be promptly closed if an I/O error occurs.

API Note:

This method is intended for simple cases where it is appropriate and convenient to read the entire input into a list of lines. It is not suitable for reading input from an unknown origin, as this may result in the allocation of an arbitrary amount of memory.

Returns:

the remaining characters as lines of text stored in an unmodifiable `List` of `Strings` in the order they are read

Throws:

`IOException` - If an I/O error occurs

`OutOfMemoryError` - If the number of remaining characters exceeds the implementation limit for `String`.

Since:

25

See Also:

`String.lines()`,
`readAllAsString()`,
`Files.readAllLines(Path, Charset)`

readAllAsString

```
public String readAllAsString()
                        throws IOException
```

Reads all remaining characters into a string. This method blocks until all remaining characters including all line separators have been read and end of stream is detected, or an exception is thrown. The resulting string will contain line separators as they appear in the stream. This method does not close the reader.

When this reader reaches the end of the stream, further invocations of this method will return an empty string.

The behavior for the case where the reader is *asynchronously closed*, or the thread interrupted during the read, is highly reader specific, and therefore not specified.

If an I/O error occurs reading from the stream then it may do so after some, but not all, characters have been read. Consequently the stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the reader be promptly closed if an I/O error occurs.

API Note:

This method is intended for simple cases where it is appropriate and convenient to read the entire input into a `String`. It is not suitable for reading input from an unknown origin, as this may result in the allocation of an arbitrary amount of memory.

Returns:

a `String` containing all remaining characters

Throws:

`IOException` - If an I/O error occurs

`OutOfMemoryError` - If the number of remaining characters exceeds the implementation limit for `String`.

Since:

25

See Also:

`readAllLines()`, `Files.readString(Path)`

skip

```
public long skip(long n)
          throws IOException
```

Skips characters. This method will block until some characters are available, an I/O error occurs, or the end of the stream is reached. If the stream is already at its end before this method is invoked, then no characters are skipped and zero is returned.

Parameters:

`n` - The number of characters to skip

Returns:

The number of characters actually skipped

Throws:

`IllegalArgumentException` - If `n` is negative.

`IOException` - If an I/O error occurs

ready

```
public boolean ready()
              throws IOException
```

Tells whether this stream is ready to be read.

Returns:

True if the next read() is guaranteed not to block for input, false otherwise. Note that returning false does not guarantee that the next read will block.

Throws:

`IOException` - If an I/O error occurs

markSupported

```
public boolean markSupported()
```

Tells whether this stream supports the mark() operation. The default implementation always returns false. Subclasses should override this method.

Returns:

true if and only if this stream supports the mark operation.

mark

```
public void mark(int readAheadLimit)
          throws IOException
```

Marks the present position in the stream. Subsequent calls to reset() will attempt to reposition the stream to this point. Not all character-input streams support the mark() operation.

Parameters:

`readAheadLimit` - Limit on the number of characters that may be read while still preserving the mark. After reading this many characters, attempting to reset the stream may fail.

Throws:

`IOException` - If the stream does not support mark(), or if some other I/O error occurs

reset

```
public void reset()
           throws IOException
```

Resets the stream. If the stream has been marked, then attempt to reposition it at the mark. If the stream has not been marked, then attempt to reset it in some way appropriate to the particular stream, for example by repositioning it to its starting point. Not all character-input streams support the reset() operation, and some support reset() without supporting mark().

Throws:

`IOException` - If the stream has not been marked, or if the mark has been invalidated, or if the stream does not support reset(), or if some other I/O error occurs

close

```
public abstract void close()
                   throws IOException
```

Closes the stream and releases any system resources associated with it. Once the stream has been closed, further read(), ready(), mark(), reset(), or skip() invocations will throw an IOException. Closing a previously closed stream has no effect.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Throws:

`IOException` - If an I/O error occurs

transferTo

```
public long transferTo(Writer out)
                throws IOException
```

Reads all characters from this reader and writes the characters to the given writer in the order that they are read. On return, this reader will be at end of the stream. This method does not close either reader or writer.

This method may block indefinitely reading from the reader, or writing to the writer. The behavior for the case where the reader and/or writer is *asynchronously closed*, or the thread interrupted during the transfer, is highly reader and writer specific, and therefore not specified.

If the total number of characters transferred is greater than Long.MAX_VALUE, then `Long.MAX_VALUE` will be returned.

If an I/O error occurs reading from the reader or writing to the writer, then it may do so after some characters have been read or written. Consequently the

reader may not be at end of the stream and one, or both, streams may be in an inconsistent state. It is strongly recommended that both streams be promptly closed if an I/O error occurs.

Parameters:

`out` - the writer, non-null

Returns:

the number of characters transferred

Throws:

`IOException` - if an I/O error occurs when reading or writing

`NullPointerException` - if `out` is `null`

Since:

10

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.